Diseño y Análisis de Algoritmos

Ayudantía 1: Divide y Conquistar

Ayud. Ferran Rojas Andreu

1. (a) Idea del algoritmo

Toda inversión de $A[lo..hi]$ cae en una de tres clases:

- ambas posiciones en la mitad izquierda;
- ambas posiciones en la mitad derecha;
- la posición i en la mitad izquierda y la j en la mitad derecha (*inversiones cruzadas*).

El algoritmo cuenta recursivamente las inversiones de cada mitad y, durante la combinación, cuenta las inversiones cruzadas: si cada mitad llega *ordenada*, al tomar un elemento de la mitad derecha que es menor que el elemento actual de la izquierda, ese elemento derecho forma una inversión con *todos* los elementos restantes de la mitad izquierda. La combinación es exactamente el `merge` de merge sort con un contador adicional.

2. (b) Pseudocódigo y complejidad

```
CountInversions(A, lo, hi)
1  if lo = hi
2      return ⟨0, A[lo]⟩
3  mid ← ⌊(lo + hi)/2⌋
4  ⟨c_L, L⟩ ← CountInversions(A, lo, mid)
5  ⟨c_R, R⟩ ← CountInversions(A, mid + 1, hi)
6  ⟨c_M, S⟩ ← MergeAndCount(L, R)
7  return ⟨c_L + c_R + c_M, S⟩
```

```
MergeAndCount(L, R)
1  i ← 0; j ← 0; c ← 0; S ← [ ]
2  while i < |L| and j < |R|
3      if L[i] ≤ R[j]
4          append L[i] a S; i ← i + 1
5      else
6          c ← c + (|L| − i)                    ▷ todas las restantes de L son >R[j]
7          append R[j] a S; j ← j + 1
8  Appendar el resto de L o de R a S
9  return ⟨c, S⟩
```

Complejidad. Cada nivel del árbol de recursión hace trabajo $O(n)$ y el problema se parte en dos mitades:

$$T(n) = 2\,T\left(\frac{n}{2}\right) + \Theta(n), \qquad T(1) = \Theta(1),$$

cuya solución es

$$\boxed{T(n) = \Theta(n \log n)}.$$

3. (c) Ejecución sobre $A = \langle 5, 3, 2, 4, 1 \rangle$

Paso 1: mitad izquierda $A[1.,2] = \langle 5, 3 \rangle$

- Subproblemas base: $\langle 5 \rangle$, $\langle 3 \rangle$ (0 inversiones c/u).
- MergeAndCount($[5], [3]$): $5 > 3$, se toma 3 de la derecha, restantes en izquierda = 1 $\Rightarrow c = 1$. Ordenada: $[3, 5]$.

$$\text{izquierda } \langle 5, 3 \rangle \Rightarrow \text{inversiones } 1, \quad \text{ordenada } [3, 5].$$

Paso 2: mitad derecha $A[3.,5] = \langle 2, 4, 1 \rangle$

- Sub-mitad $\langle 2, 4 \rangle$: bases $[2]$, $[4]$; MergeAndCount: $2 \leq 4$, sin inversiones. Ordenada $[2, 4]$. Inversiones 0.
- Sub-mitad $\langle 1 \rangle$: 0 inversiones, ordenada $[1]$.
- MergeAndCount($[2, 4], [1]$): $2 > 1 \Rightarrow c = 2$ (los elementos 2 y 4 forman inversión con 1); luego se agrega $2, 4$. Ordenada $[1, 2, 4]$. Inversiones 2.

$$\text{derecha } \langle 2, 4, 1 \rangle \Rightarrow \text{inversiones } 2, \quad \text{ordenada } [1, 2, 4].$$

Paso 3: combinación de $[3, 5]$ y $[1, 2, 4]$

Comparación	Acción	c acumulado
3 vs 1	tomar 1 (derecha); restantes en $L = 2$ $(3, 5)$	$+2 \Rightarrow 2$
3 vs 2	tomar 2 (derecha); restantes en $L = 2$	$+2 \Rightarrow 4$
3 vs 4	tomar 3 (izquierda)	4
5 vs 4	tomar 4 (derecha); restantes en $L = 1$ (5)	$+1 \Rightarrow 5$

$$\text{cruzadas } = 5, \qquad \text{arreglo final ordenado } [1, 2, 3, 4, 5].$$

Resultado

$$\boxed{\text{Inversiones} = 1 + 2 + 5 = 8}.$$

Verificación por fuerza bruta (pares $i < j$ con $A[i] > A[j]$):

$$(5,3), (5,2), (5,4), (5,1), \ (3,2), (3,1), \ (2,1), \ (4,1) \quad \Rightarrow 8.$$

4. (a) Idea del algoritmo

El conjunto de candidatos es el intervalo de enteros $[lo, hi] = [0, n]$. El algoritmo mantiene la invariante

$$\lfloor \sqrt{n} \rfloor \in [lo, hi],$$

y reduce el intervalo a la mitad en cada paso, como una búsqueda binaria:

- si $m^2 \le n$, entonces $m \le \sqrt{n}$ y la raíz buscada está en $[m, hi]$; se descarta la mitad inferior;
- si $m^2 > n$, entonces $m > \sqrt{n}$ y la raíz está en $[lo, m-1]$; se descarta la mitad superior.

Usar el punto medio *redondeado hacia arriba* $m = \lfloor (lo + hi + 1)/2 \rfloor$ garantiza que lo crece y que, al terminar, el valor retenido es el mayor m con $m^2 \le n$.

5. (b) Pseudocódigo y complejidad

```
FloorSqrt(n)
1  lo ← 0; hi ← n
2  while lo < hi
3      m ← ⌊(lo + hi + 1)/2⌋
4      if m · m ≤ n
5          lo ← m
6      else
7          hi ← m − 1
8  return lo
```

Complejidad. Sea N el tamaño del intervalo inicial. En cada iteración el intervalo se reduce a la mitad, por lo que el número de iteraciones satisface

$$T(N) = T\left(\frac{N}{2}\right) + \Theta(1), \qquad T(1) = \Theta(1),$$

cuya solución es

$$\boxed{T(N) = \Theta(\log N) = \Theta(\log n)}.$$

El espacio adicional es $\Theta(1)$ (solo lo, hi y m).

6. (c) Ejecución sobre $n = 17$

lo	hi	$m = \lfloor (lo + hi + 1)/2 \rfloor$	m^2 vs 17	Acción
0	17	9	$81 > 17$	$hi \leftarrow 8$
0	8	4	$16 \le 17$	$lo \leftarrow 4$
4	8	6	$36 > 17$	$hi \leftarrow 5$
4	5	5	$25 > 17$	$hi \leftarrow 4$

Como ahora $lo = hi = 4$, el bucle termina y se retorna:

$$\boxed{\lfloor \sqrt{17} \rfloor = 4}.$$

Verificación: $4^2 = 16 \le 17$ y $5^2 = 25 > 17$, luego 4 es el mayor entero con $m^2 \le 17$. ✓